

Challenging Questions

Applied Data Science - Assignment 2

Mahshid Bargahi

May 19, 2026

1- Explain bias-variance trade-off in regression models.

The bias-variance trade-off in regression describes the tension between two sources of error that affect model performance.

The total expected error of a regression model can be decomposed as:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error} \quad (1)$$

where:

- **Bias:** Error due to overly simplistic assumptions. High bias leads to underfitting.

$$\text{Bias}[\hat{f}(X)] = E[\hat{f}(X)] - f(X)$$

- **Variance:** Error due to sensitivity to small fluctuations in training data. High variance leads to overfitting.

$$\text{Var}[\hat{f}(X)] = E[(\hat{f}(X) - E[\hat{f}(X)])^2]$$

- **Irreducible Error:** Noise inherent in the data that cannot be eliminated.

Relationship with Model Complexity

Model Complexity	Bias	Variance
Too Simple (Underfitting)	High	Low
Optimal	Low	Low
Too Complex (Overfitting)	Low	High

Because minimizing one typically increases the other, the goal is to find the optimal model complexity that balances both to minimize the total prediction error on new, unseen data.

2- When Does Kernel Regression Outperform Linear Regression?

Kernel regression outperforms linear regression when:

1. The relationship between features and target is **non-linear**
2. The data has **complex patterns** that cannot be captured by a straight line
3. The feature space is **low-dimensional** (kernel regression suffers from curse of dimensionality)

5- Explain the effect of class imbalance on binary metrics. Why is accuracy misleading in imbalanced datasets?

Class imbalance occurs when one class (the majority) significantly outnumbers the other (the minority).

From the confusion matrix:

	Predicted: Positive	Predicted: Negative
Actual: Positive	TP (True Positive)	FN (False Negative)
Actual: Negative	FP (False Positive)	TN (True Negative)

- **Accuracy:**

$$\text{Accuracy} = \frac{TP + TN}{\text{Total Samples}} \quad (2)$$

Accuracy becomes misleadingly high in imbalanced datasets because it counts all correct predictions equally.

- **Precision:**

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3)$$

Precision measures how many predicted positive cases are actually positive. In imbalanced datasets, precision for the minority class can be very low if the model produces many false positives. Conversely, precision for the majority class tends to be high because false positives for that class are rare. Precision alone does not reveal whether the model is missing minority instances.

- **Recall (Sensitivity):**

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4)$$

Recall measures the proportion of actual positive (minority) cases that were correctly identified. In imbalanced datasets, recall for the minority class is often poor because the model tends to predict the majority class by default, leading to many false negatives. Low recall means the model fails to detect the rare but critical events (e.g., fraud, disease).

- **Specificity:**

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (5)$$

Specificity measures the proportion of actual negative (majority) cases correctly identified. In imbalanced datasets, specificity is typically very high because the model easily predicts the majority class correctly. However, high specificity does not imply good overall performance — it can be achieved while recall on the minority class is near zero.

The Problem with Accuracy

In imbalanced datasets, a model can achieve high accuracy by simply predicting the majority class for all samples.

Actual	Predicted: Positive	Predicted: Negative
Positive (1%)	0	10
Negative (99%)	0	990

$$\text{Accuracy} = \frac{990}{1000} = 99\% \quad (\text{misleading!}) \quad (6)$$

$$\text{Recall} = \frac{0}{10} = 0 \quad (\text{model fails completely!}) \quad (7)$$

Why Accuracy Fails?

Accuracy does not distinguish between correct predictions of majority vs. minority classes. A dummy classifier predicting only the majority class achieves high accuracy while being practically useless.

8- Overfitting in Decision Trees:

• Why Do Decision Trees Overfit Easily?

Decision trees are prone to overfitting because they can keep splitting the data until every leaf node is perfectly pure (contains samples from only one class or has zero variance in regression). This extreme flexibility allows them to memorize the training data, including its noise and outliers, rather than learning the true underlying pattern.

Reasons:

- **High model flexibility :** Decision trees make no assumptions about the data distribution. They can create highly complex decision boundaries that bend to fit every training point.
- **Deep trees capture noise :** With enough depth, the tree can split until each leaf contains a single sample — perfectly fitting the training set but failing on new data.
- **No inherent regularization**
- **Sensitivity to outliers :** A single outlier can cause a deep split that isolates it, creating a rule that generalizes poorly.
- **Instability (high variance) :** Small changes in training data (e.g., removing one sample) can produce a completely different tree structure, a sign of overfitting.
- **Multiple testing effect :** At each split, the tree tests many features and thresholds. With many splits, it will eventually find spurious patterns that exist only by chance in the training data.

Shallow tree (depth 2-3): Underfits, high bias, but generalizes better.

Deep tree (depth 20): Perfect training accuracy, but test accuracy drops sharply — classic overfitting.

- **Why is max_depth Alone Not Enough?**

Even with a fixed `max_depth`, a tree can still overfit because:

- A depth-5 tree can still have leaves with only 1-2 samples
- It doesn't control the minimum samples in leaves
- It doesn't prevent unnecessary splits within the depth limit

Effective regularization requires a combination of parameters.

Additional Regularization Parameters

- `min_samples_split`: Minimum samples required to split an internal node
- `min_samples_leaf`: Minimum samples in a leaf node
- `max_leaf_nodes`: Maximum number of leaf nodes
- `min_impurity_decrease`: Minimum impurity reduction to allow a split

- **How Pruning Works?**

Pruning removes branches that have little statistical importance.

Cost-Complexity Pruning (CCP) :

$$R_{\alpha}(T) = R(T) + \alpha \times |T| \quad (8)$$

where:

- $R(T)$ = misclassification error on training data
- $|T|$ = number of leaf nodes
- α = complexity parameter (higher α = more pruning)

Pruning Algorithm:

1. Grow a full, unpruned tree
2. For each internal node, compute the cost-complexity measure
3. Find the weakest link (node with smallest increase in error per leaf removed)
4. Prune that node, repeat until a single node remains
5. Use cross-validation to select the optimal tree size

Before Pruning	After Pruning
complex tree with many leaves	simpler tree with fewer leaves

9- Explain why Tree-based models are good feature selectors

Tree-based models naturally perform feature selection as part of their training process. They evaluate features repeatedly and rank them by how useful they are for splitting the data.

Reasons :

- **Built-in Feature Importance :**

Trees compute importance scores (e.g., Gini importance, permutation importance) that rank features by their contribution to reducing impurity.

Gini Importance / Mean Decrease in Impurity:

$$\text{Importance}(f) = \sum_{\text{nodes } n \text{ that split on } f} \frac{N_n}{N} \times \Delta I_n \quad (9)$$

where ΔI_n is the reduction in impurity at node n , and N_n is the number of samples reaching that node.

Feature Importance Example

For a dataset with 127 features, a Random Forest ranks them:

Rank	Feature	Importance
1	Price	0.45
2	Area	0.23
3	Number of Rooms	0.12
4	Location Score	0.08
$\vdots$	$\vdots$	$\vdots$
127	Irrelevant Feature	0.00

- **Handles irrelevant features :** Irrelevant features are rarely selected for splits, effectively ignoring them.
- **Detects non-linear relationships :** Unlike linear models (which need transformations), trees can capture complex, non-linear patterns without manual feature engineering.
- **Robust to outliers and scaling :** Tree splits are based on thresholds, so feature scaling and outliers have minimal impact on selection.

Comparison with Other Models

Model	Inherent Feature Selection?	Interpretability
Decision Tree / Random Forest	Yes	High
XGBoost / LightGBM	Yes	Medium
Logistic Regression	Partial (coefficients)	Medium
SVM	No	Low
KNN	No	Low
Neural Networks	No	Low

12- Explain the precision-recall trade-off.

Precision and recall are two metrics that often conflict with each other. Improving one typically worsens the other, forcing you to choose a balance based on your specific problem.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (\text{Of predicted positives, how many are correct?}) \quad (10)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (\text{Of actual positives, how many did we find?}) \quad (11)$$

The Trade-off

Precision and recall have an inverse relationship:

$$\text{Precision} \uparrow \implies \text{Recall} \downarrow \quad (12)$$

$$\text{Precision} \downarrow \implies \text{Recall} \uparrow \quad (13)$$

You cannot simultaneously maximize both.

Threshold as a Control Lever

Models that output probabilities allow adjusting the decision threshold:

Threshold	Precision	Recall	Interpretation
0.9 (High)	Very High	Very Low	Only very confident predictions
0.5 (Default)	Balanced	Balanced	Default trade-off
0.1 (Low)	Low	Very High	Capture all possible positives

13- Explain ROC vs PR curve

Both curves evaluate binary classifiers across all possible decision thresholds, but they focus on different aspects of performance.

- **ROC(Receiver Operating Characteristic) curve:**

ROC curves plot True Positive Rate against False Positive Rate and are excellent for evaluating a classifier's ability to separate classes, but they can be overly optimistic in imbalanced datasets because they include true negatives in the calculation.

- **PR(Precision-Recall) curve:**

PR curves plot Precision against Recall, focusing exclusively on the positive (typically minority) class, making them much more informative for severe class imbalance (e.g., fraud detection, disease screening).

	ROC Curve	PR Curve
X-axis	False Positive Rate (FPR)	Recall
Y-axis	True Positive Rate (TPR/Recall)	Precision

where:

$$\mathbf{FPR} = \frac{FP}{FP+TN}$$

When to Use	ROC Curve	PR Curve
Balanced Data	Good	Good
Imbalanced Data	May be optimistic	Preferred
Class Distribution	Assumes balanced	Handles imbalance well

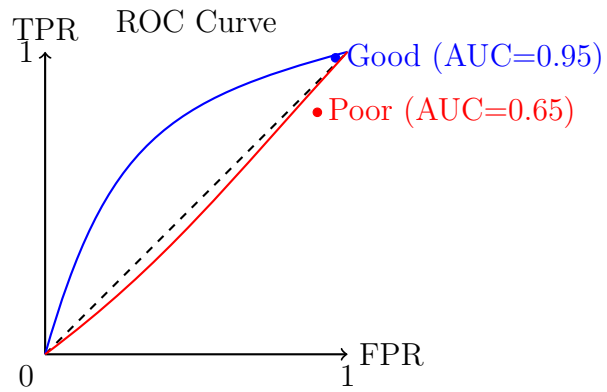


Figure 1: ROC Curve

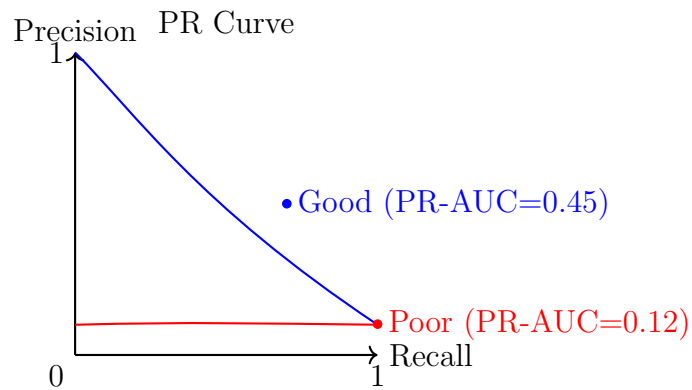


Figure 2: PR Curve